

List-Based Scheduling for High-Level Synthesis

Md Azadul Islam

Student ID: 2220070

HW/SW Co-design

Hamm-Lippstadt University of Applied Science

Seminar Paper

Abstract—High-Level Synthesis (HLS) translates an algorithmic description into register-transfer level (RTL) hardware. One of the central HLS tasks is scheduling, where operations are assigned to control steps while respecting data dependencies and resource constraints. This paper studies List-Based Scheduling as a practical heuristic for resource-constrained scheduling. A small arithmetic example is implemented first as a C++ scheduler and then as two VHDL RTL realizations. The experiments compare a design with one arithmetic logic unit (ALU) and one multiplier against a design with two ALUs and one multiplier. The C++ scheduler reports a latency of 4 cycles for the one-ALU design and 3 cycles for the two-ALU design. QuestaSim verification confirms that both RTL designs compute the correct result, $z = 46$, while the two-ALU design finishes earlier. The results demonstrate the typical HLS tradeoff between hardware resources and latency.

Index Terms—High-Level Synthesis, List-Based Scheduling, RTL, VHDL, resource constraints, latency, C++.

I. INTRODUCTION

Modern embedded and digital systems often require a compromise between execution speed, hardware area, and design cost. High-Level Synthesis (HLS) supports this process by translating an algorithmic description, for example written in C or C++, into an RTL-level hardware implementation. Teich and Haubelt describe synthesis as an automated design process and present allocation, scheduling, and binding as fundamental synthesis tasks [1]. De Micheli also treats scheduling and binding as important problems in architectural-level synthesis [2].

This paper focuses on scheduling. Scheduling decides in which clock cycle or control step each operation is executed. If hardware resources are limited, independent operations still may not be able to run in the same cycle. List-Based Scheduling addresses this by maintaining a list of ready operations and selecting operations according to a priority rule while checking resource availability.

The contribution of this work is an educational implementation study. A C++ program is used to produce schedules for two resource configurations. Then the same schedules are realized in VHDL and verified in QuestaSim. This connects the HLS scheduling concept to concrete RTL behavior.

II. BACKGROUND

In HLS, an algorithm is mapped to hardware components such as ALUs, multipliers, registers, multiplexers, and a control unit. Three synthesis terms are especially relevant:

- *Allocation*: decides the available hardware resources, such as one ALU or two ALUs.

- *Scheduling*: assigns operations to clock cycles or control steps.
- *Binding*: maps operations to concrete hardware units.

Teich and Haubelt discuss these synthesis tasks independently of the final implementation type and describe operations such as additions and multiplications as tasks that are mapped to resources such as ALUs and multipliers [1]. This is directly related to the example in this paper, where addition and multiplication operations are scheduled under ALU and multiplier constraints.

III. MATHEMATICAL MODEL OF THE EXAMPLE

The arithmetic expression used in the experiment is

$$z = (a + b)(c + d) + e. \quad (1)$$

It is decomposed into four operations:

$$v_1 = a + b, \quad (2)$$

$$v_2 = c + d, \quad (3)$$

$$v_3 = v_1 \times v_2, \quad (4)$$

$$v_4 = v_3 + e. \quad (5)$$

The operation graph is represented as $G = (V, E)$, where V is the set of operations and E is the set of dependency edges:

$$V = \{v_1, v_2, v_3, v_4\}, \quad (6)$$

$$E = \{(v_1, v_3), (v_2, v_3), (v_3, v_4)\}. \quad (7)$$

The edges mean that v_3 cannot start before both v_1 and v_2 finish, and v_4 cannot start before v_3 finishes. If $s(v_i)$ denotes the start cycle of operation v_i and $d(v_i)$ its duration, the dependency constraint is

$$s(v_j) \geq s(v_i) + d(v_i), \quad \forall (v_i, v_j) \in E. \quad (8)$$

The resource constraint states that the number of operations of a certain type scheduled in a cycle must not exceed the number of available resources. For example, with one ALU, two additions cannot execute in the same cycle.

IV. LIST-BASED SCHEDULING ALGORITHM

List-Based Scheduling is a constructive heuristic. It maintains a ready list of operations whose predecessor operations have already been completed. The ready operations are sorted by priority, and the scheduler assigns operations to the current cycle if the required resource is available. De Micheli places

list scheduling among heuristic scheduling algorithms for resource-constrained scheduling [2]. Teich and Haubelt also present list scheduling in the context of resource-constrained scheduling [1].

The simplified algorithm implemented in this work is:

```

Input: operation graph G(V,E), resource limits
Output: start cycle for every operation

while not all operations are scheduled:
    ready_list = operations whose dependencies are
                finished
    sort ready_list by priority
    for each operation in ready_list:
        if required resource is available:
            schedule operation in current cycle
            mark resource as used
        move to next cycle

```

The method is fast and practical for educational and many engineering cases. However, it is a heuristic and therefore does not guarantee a globally optimal schedule for every possible problem.

V. C++ IMPLEMENTATION

The C++ implementation represents each operation using a structure containing its name, type, duration, priority, dependencies, and scheduled start cycle. The function `dependenciesFinished()` checks whether all predecessors of an operation have already completed. In every cycle, the program builds the ready list, sorts it by priority, and schedules operations if the required resource is available.

A. One ALU and One Multiplier

In the first experiment, the resources are one ALU and one multiplier. Since $v_1 = a + b$ and $v_2 = c + d$ both require an ALU, they must be scheduled in separate cycles. The resulting schedule is shown in Table I and the C++ output is shown in Fig. 1.

TABLE I
SCHEDULE WITH ONE ALU AND ONE MULTIPLIER.

Cycle	Scheduled operation
1	$v_1 = a + b$
2	$v_2 = c + d$
3	$v_3 = v_1 \times v_2$
4	$v_4 = v_3 + e$

B. Two ALUs and One Multiplier

In the second experiment, two ALUs and one multiplier are available. The two independent additions can therefore be executed in parallel in cycle 1. The resulting schedule is shown in Table II and the C++ output is shown in Fig. 2.

VI. VHDL RTL REALIZATION AND SIMULATION

The VHDL implementation realizes the scheduled operations at RTL level. Two modules were created: `hls_sched_1alu.vhd` for the 4-cycle schedule and

```

PS C:\Users\User\Downloads\achim\cpp> .\List_scheduler.exe
List-Based Scheduling for High-Level Synthesis
-----

Resource Constraints:
ALU units      : 1
Multiplier units : 1

Scheduled Output:
-----
Cycle 1: v1 = a + b [ALU]
Cycle 2: v2 = c + d [ALU]
Cycle 3: v3 = v1 * v2 [MUL]
Cycle 4: v4 = v3 + e [ALU]

Final Operation Start Times:
-----
v1 = a + b starts at cycle 1
v2 = c + d starts at cycle 2
v3 = v1 * v2 starts at cycle 3
v4 = v3 + e starts at cycle 4

Total latency = 4 cycles

```

Fig. 1. C++ List-Based Scheduling output with one ALU and one multiplier. The two addition operations are scheduled in separate cycles, resulting in a latency of 4 cycles.

TABLE II
SCHEDULE WITH TWO ALUS AND ONE MULTIPLIER.

Cycle	Scheduled operation
1	$v_1 = a + b$ and $v_2 = c + d$
2	$v_3 = v_1 \times v_2$
3	$v_4 = v_3 + e$

`hls_sched_2alu.vhd` for the 3-cycle schedule. The testbench applies the input values

$$a = 2, \quad b = 3, \quad c = 4, \quad d = 5, \quad e = 1. \quad (9)$$

The expected result is

$$z = (2 + 3)(4 + 5) + 1 = 5 \cdot 9 + 1 = 46. \quad (10)$$

The QuestaSim waveform in Fig. 3 shows that both designs produce $z = 46$. The signal `done_2alu` becomes high earlier

```

List-Based Scheduling for High-Level Synthesis
-----

Resource Constraints:
ALU units      : 2
Multiplier units : 1

Scheduled Output:
-----
Cycle 1: v1 = a + b [ALU], v2 = c + d [ALU]
Cycle 2: v3 = v1 * v2 [MUL]
Cycle 3: v4 = v3 + e [ALU]

Final Operation Start Times:
-----
v1 = a + b starts at cycle 1
v2 = c + d starts at cycle 1
v3 = v1 * v2 starts at cycle 2
v4 = v3 + e starts at cycle 3

Total latency = 3 cycles

```

Fig. 2. C++ List-Based Scheduling output with two ALUs and one multiplier. The two independent addition operations execute in parallel in cycle 1, reducing the latency to 3 cycles.

than `done_1alu`, confirming that the two-ALU implementation finishes earlier.

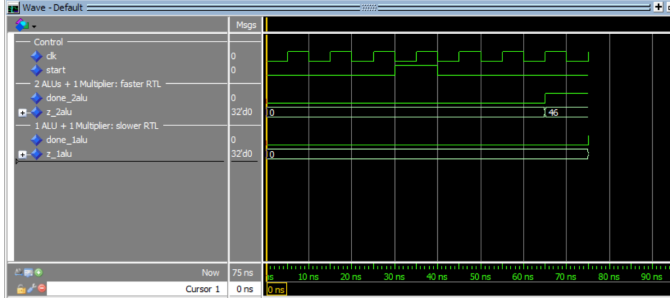


Fig. 3. QuestaSim waveform of the VHDL RTL implementations. Both designs compute the same result $z = 46$. The two-ALU design finishes earlier than the one-ALU design, confirming the resource-latency tradeoff.

VII. RESULTS AND DISCUSSION

Table III summarizes the results. The C++ program demonstrates the scheduling decisions, while the VHDL simulation confirms the RTL behavior.

TABLE III
COMPARISON OF RESOURCE CONFIGURATIONS.

Configuration	Behavior	Latency	Output
1 ALU + 1 multiplier	Sequential additions	4 cycles	46
2 ALUs + 1 multiplier	Parallel additions	3 cycles	46

The results show the main tradeoff in resource-constrained HLS scheduling. With one ALU, operations v_1 and v_2 cannot run together, so the schedule requires four cycles. With two ALUs, these independent additions can run in parallel, reducing the latency to three cycles. This improvement requires additional hardware, so the design trades area for performance.

VIII. LIMITATIONS

The implementation is intentionally small and educational. It does not include register allocation optimization, memory access scheduling, multiplexer sharing, power estimation, or automatic priority computation. Larger designs may require more advanced heuristics or exact optimization methods. Nevertheless, the example demonstrates the essential idea of List-Based Scheduling and its connection to RTL implementation.

IX. CONCLUSION

This paper presented List-Based Scheduling for High-Level Synthesis using a simple arithmetic data-flow example. A C++ implementation generated schedules under two resource configurations, and VHDL RTL simulations verified the resulting hardware behavior. Increasing the available ALUs from one to two reduced latency from 4 cycles to 3 cycles, while both designs produced the correct output, $z = 46$. The experiment demonstrates how List-Based Scheduling exploits parallelism when resources are available and highlights the HLS tradeoff between latency and hardware cost.

AI USAGE STATEMENT

AI assistance was used to organize the explanation, prepare the LaTeX draft, and improve wording. The C++ outputs and VHDL waveform included in this paper were generated from the author's implementation and simulation runs. The final technical content should be checked against the cited references and submitted according to the course rules.

REFERENCES

- [1] J. Teich and C. Haubelt, *Digitale Hardware/Software-Systeme: Synthese und Optimierung*, 2nd ed. Berlin, Germany: Springer, 2007.
- [2] G. De Micheli, *Synthesis and Optimization of Digital Circuits*. New York, NY, USA: McGraw-Hill, 1994.